

AI Research Assistant - Project

Day - 17

1. Objective

The primary objective for Day 2 was to transition the application from a static frontend prototype into a dynamic, multi-user platform. This involved implementing a complete user authentication system, which required establishing a database, writing the backend logic for user registration and login, and ensuring user sessions are managed securely. The goal was to create a secure gateway to the application's core features.

2. Theoretical Concepts Applied

- **Database Management System (DBMS):** A DBMS is a software system used to create, manage, and query databases. For this project, **SQLite** was chosen. Unlike larger database systems like PostgreSQL or MySQL that run as separate server processes, SQLite is a serverless, self-contained, and transactional SQL database engine. This makes it incredibly lightweight and easy to integrate directly into a Python application, as the entire database is stored in a single file (database.db). This is ideal for development, prototyping, and small to medium-sized applications.
- **CRUD Operations:** This acronym stands for the four fundamental functions of persistent storage: **Create**, **Read**, **Update**, and **Delete**. we implemented the first two:
 - **Create:** The registration functionality, where a new user record is created and inserted into the users table.
 - **Read:** The login functionality, where the application reads (queries) the users table to find a matching username and verify its password.
- **State Management & JSON Web Tokens (JWT):** Web applications built on protocols like HTTP are inherently **stateless**, meaning the server does not remember any information about a user between requests. To solve this, we need a session management system. Instead of using traditional server-side sessions (which would make our server stateful), we opted for **JSON Web Tokens (JWT)**, a modern, stateless approach.
 - A JWT is a compact, URL-safe means of representing claims to be transferred between two parties. When a user logs in, the server generates a signed JWT containing a payload (e.g., the user's username) and sends it to the client. The client then includes this token in the header of every subsequent request.
 - The server can verify the token's signature to confirm its authenticity without needing to store any session information, thus keeping the application stateless and highly scalable.

3. Key Activities & Implementation Details

- **Database Schema Design and Initialization:**
 - A Python function, `init_db()`, was created to handle the initial database setup.
 - Using the `sqlite3` module in Python, an SQL `CREATE TABLE IF NOT EXISTS` statement was executed to define the schema for the users table. This table includes three columns:
 - `id`: An `INTEGER PRIMARY KEY AUTOINCREMENT` to provide a unique identifier for each user.
 - `username`: A `TEXT NOT NULL UNIQUE` field to store the user's name, with a `UNIQUE` constraint to prevent duplicate accounts.
 - `password`: A `TEXT NOT NULL` field to store the user's password.
- **Registration Route (`/register`):**
 - The backend logic for the `POST` method of this route was implemented.
 - The function retrieves the username and password from the incoming form data (`request.form`).
 - It executes an SQL `INSERT INTO` query to add the new user to the users table.
 - **Error Handling:** The entire database operation was wrapped in a `try...except` block to gracefully handle `sqlite3.IntegrityError`. This error is automatically raised by the database if a user tries to register with a username that already exists, allowing us to flash a user-friendly error message.
- **Login Route (`/`):**
 - The backend logic for the `POST` method of the login route was developed.
 - It executes an SQL `SELECT * FROM users WHERE username=?` query to find a user in the database.
 - It then performs a direct comparison between the password submitted in the form and the password retrieved from the database (`user[2]`).
- **Secure Session Implementation:**
 - The `Flask-JWT-Extended` library was installed and initialized.
 - Upon a successful password verification, the `create_access_token(identity=username)` function is called to generate a secure JWT.
 - This token is then attached to the user's browser cookies using the `set_access_cookies()` function, which prepares the user for authenticated access to protected pages.

- A placeholder /dashboard route was created and protected with the `@jwt_required()` decorator. This decorator acts as a security checkpoint, automatically verifying the JWT in the user's cookies on every request. If the token is missing or invalid, it denies access.

4. Outcome

By the end of Day 2, the AI Research Assistant has evolved from a static website into a fully functional, dynamic web application with a robust and secure authentication system. The application can now manage user accounts, persist data in a database, and protect its core features from unauthorized access, setting the stage for the implementation of the main AI functionalities.